Optimistic UI (Like / Comment with Rollback)

Problem: Build a feed where liking a post and adding a comment update the UI instantly, before the server answers, and roll back cleanly when the request fails.
Often asked at: Meta, LinkedIn
Tech stack: React (Vite, JavaScript, Tailwind) · **Time limit:** 60 minutes

---

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what must it do, what is out of scope.
2. **List interactions**: every user action and the expected result. This becomes your checklist.
3. **Sketch the architecture**: components, hooks, services, and who talks to whom.
4. **Design the state**: the minimum state, who owns it, what is derived.
5. **Define data and contracts**: example data, API shape, mocks.
6. **Trace one flow end to end** before writing code.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers**: a working skeleton first, then one feature at a time, always runnable.
9. **Cover states and edge cases**: loading, empty, error, success, and the problem's own states.
10. **Wrap up**: say what is done, the trade-offs, and what you would add with more time.

Time budget for 60 minutes

Minutes	Activity
0 – 7	Requirements, clarifying questions, assumptions
7 – 15	Interactions, state model, API contract, rollback strategy
15 – 25	Fake API + reducer + feed rendering (static working version)
25 – 38	Optimistic like with rollback and click coalescing
38 – 50	Optimistic comment with pending / failed / retry / discard
50 – 56	Loading/error states, toast, accessibility
56 – 60	Walk through edge cases and trade-offs

Think out loud

Say every decision before you make it: *"I'll keep the server's confirmed value next to the optimistic one, so a rollback never has to guess."* The interviewer grades how you reason about failure and races more than the CSS. If you are stuck, say what you are trying and why.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a feed of posts: author, text, like count, liked state, comments.
- **Like / unlike** a post: the heart and count change **instantly**.
- If the like request fails: **roll back** to the last confirmed value and tell the user.
- **Add a comment**: it appears **instantly** in the list, marked as "Sending…".
- If the comment request fails: keep the comment, mark it **Failed**, offer **Retry** and **Discard**.
- A way to simulate failures so the rollback can be demonstrated.

Non-functional requirements

Area	Requirement
Performance	Zero perceived latency on like/comment; no request storm on rapid clicks
Correctness	UI ends in the same state as the server, even with rapid clicks and out-of-order responses
Error handling	Never lose what the user typed; clear, non-blocking error message
Accessibility	`aria-pressed` on the like toggle, labelled inputs, failures announced through an `aria-live` region
Reusability	The API layer and the reducer are plain JS, independent of components
Extensibility	Same pattern works for bookmarks, reactions, follows, edits and deletes

Clarifying questions

1. Should the like button stay clickable while a request is in flight, or be disabled?
2. When a comment fails, should it disappear, or stay with a retry option?
3. Does the API toggle the like, or accept the desired state (`liked: true/false`)?
4. Can the server assign comment ids, and does it accept a client-generated id for de-duplication?
5. Do we need real-time updates from other users (likes by others changing the count)?
6. How should errors be shown: inline, toast, or both?

Assumptions

- Like button is **never disabled**; rapid clicks are coalesced (only the final intent is sent).
- API is **idempotent**: `setLike(postId, liked)` sets the desired state; `addComment` takes a `clientId`.
- Failed comments **stay** with Retry / Discard; failed likes **roll back** with a toast.
- Single current user, no real-time sync, no pagination (a few posts).
- Fake API with 400–1200 ms latency and a 30% failure rate that the UI can switch off.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	"Loading posts…", then the feed
Initial load fails	Error message + Retry button
Click ♡ (not liked)	Instantly ♥, count +1, "saving…" shown; server confirms, "saving…" disappears
Click ♥ (liked)	Instantly ♡, count −1
Like request fails	Heart and count return to last confirmed value; toast "Couldn't update like"
Click like 5 times quickly	UI flips every click; at most 2 requests go out; final state = final click
Like then unlike before the response	UI shows unliked; after the first response, one more request sends `false`
Type comment + Post	Comment appears at once, faded, "Sending…"; input clears
Comment request succeeds	Comment becomes normal; temp entry gets the real server id
Comment request fails	Comment stays, red "Failed", Retry and Discard buttons, toast
Click Retry	Status back to "Sending…", same `clientId` resent (no duplicate on server)
Click Discard	Failed comment is removed
Submit empty / spaces only	Post button disabled, nothing happens
Toggle "Simulate failures" off	All requests succeed

3. Design the architecture

Module	Responsibility
`api/fakeApi.js`	Fake server: in-memory DB, latency, failure rate, idempotent endpoints
`state/feedReducer.js`	Pure state transitions: load, optimistic set, sync, rollback, comment statuses
`hooks/useFeed.js`	Owns the reducer; runs requests; coalesces likes; triggers rollback and toasts
`App.jsx`	Page layout, loading / error / empty states, failure toggle
`PostCard.jsx`	One post: text, like button, comments, form
`LikeButton.jsx`	Heart + derived count + "saving…" indicator
`CommentList.jsx`	Comments with pending / failed / retry / discard UI
`CommentForm.jsx`	Controlled input, trims, clears optimistically
`Toast.jsx`	Auto-hiding message in an `aria-live` region

```
App
├── useFeed()  ── feedReducer (pure)
```

```
│      └── api (fakeApi)
├── PostCard (×N)
│   ├── LikeButton
│   ├── CommentList
│   └── CommentForm
└── Toast
```

Start simple: a single `App` with the reducer is enough for the first pass. Split into `useFeed` once the request logic grows, and into small components once the like and comment UI have their own states.

---

4. Design the state / data model

State or field	Purpose
`status`	`'loading' | 'ready' | 'error'` for the initial load
`error`	Load error message
`posts[]`	The feed, in display order
`post.liked`	What the user **sees** (optimistic)
`post.server`	`{ liked, likes }`: last state the **server confirmed**
`post.comments[]`	`{ id, clientId, text, status }`
`comment.status`	`'pending' | 'sent' | 'failed'`
`intent` (ref)	postId → liked value the user wants right now
`inFlight` (ref)	postIds with a like request on the wire
`notice`	`{ id, text }` for the toast

```
const initialState = { status: 'loading', posts: [], error: null };

const post = {
  id: 'p1', author: 'Asha', text: '…',
  liked: true,                          // optimistic, what the UI shows
  server: { liked: false, likes: 12 },  // last confirmed by the server
  comments: [{ id: null, clientId: 'b1f…', text: 'hi', status: 'pending' }],
};
```

Senior notes

- **Derive, don't store, the like count.** `likes = server.likes ± 1` when `liked !== server.liked`. Storing a separate optimistic count invites drift (+1 twice, −1 once…). Same for "saving…": it is just `liked !== server.liked`.
- **Keep the confirmed value next to the optimistic one.** Rollback becomes `liked = server.liked`: no snapshot, no guessing.

- **Ownership:** the feed state lives in `useFeed` (one owner). The input text is local to `CommentForm`. `intent` and `inFlight` are refs: they coordinate requests and must not trigger renders.

---

5. Design the data structures and contracts

Example data

```
[
  { id: 'p1', author: 'Asha', text: 'Shipped optimistic likes today 🚀', liked: false, likes: 12,
    comments: [{ id: 'c1', clientId: 'seed-1', text: 'Nice work!' }] },
  { id: 'p2', author: 'Ravi', text: 'Rollback is the hard part…', liked: true, likes: 3, comments: []
},
]
```

Stable ids matter: React keys, reducer lookups, and matching a server response to the optimistic item all depend on them. A new comment has **no server id yet**, so the client creates a `clientId` (`crypto.randomUUID()`) and uses it as the key and as the match key for the response.

API contracts

Call	Input	Output
`getPosts()`	–	`Post[]`
`setLike(postId, liked)`	desired state	`{ liked, likes }` (server truth)
`addComment(postId, clientId, text)`	client id + text	`{ id, clientId, text }`

```
// PUT /posts/p1/like   { "liked": true }       → 200 { "liked": true, "likes": 13 }
// POST /posts/p1/comments { "clientId": "b1f…", "text": "hi" }
//                                              → 201 { "id": "c9", "clientId": "b1f…", "text": "hi" }
// Any call may fail                            → throws Error('Network error')
```

Why `setLike(liked)` and not `toggleLike()`: a toggle is **not idempotent**. If a request times out and you retry, a toggle may flip twice. Sending the desired state makes retries and duplicates harmless. The same reason for `clientId` on comments: the server de-duplicates a retried comment.

---

6. Trace the data flow

Like, then the request fails:

```
User clicks ♡ on p1
   ↓
```

```
toggleLike(post): liked = !(intent ?? post.liked) = true; intent[p1] = true
   ↓
dispatch LIKE_SET → post.liked = true
   ↓
Render: ♥ 13 "saving…"   (count derived: server.likes 12 + 1)
   ↓
inFlight has no p1 → syncLike(p1) → api.setLike(p1, true)  … 800 ms …
   ↓
Request throws
   ↓
delete intent[p1]; dispatch LIKE_ROLLBACK → post.liked = post.server.liked (false)
   ↓
notify("Couldn't update like") → Toast in aria-live region
   ↓
Render: ♡ 12   (no "saving…": liked === server.liked)
```

Success path: `setLike` resolves `{ liked: true, likes: 13 }` → `LIKE_SYNC` stores it in `post.server` → `liked === server.liked`, "saving…" disappears, count still 13.

---

7. Important technical challenges

Challenge 1: Rolling back to the right value

Problem: The common answer is "save the previous value, restore it on error". Scenario: count is 12. Click like (snapshot 12, shows 13). Before it fails, a comment request succeeds and refetches the post with 14 likes from others. Restoring the snapshot writes back a stale 12 and throws away good data. Snapshots also break when two operations on the same item overlap.

Solution: Store the **server-confirmed value** separately and only ever set the optimistic field. Rollback = copy confirmed into optimistic. The count is derived, so it always matches.

```
case 'LIKE_SET':      return updatePost(state, id, (p) => ({ ...p, liked: action.liked }));
case 'LIKE_SYNC':     return updatePost(state, id, (p) => ({ ...p, server: action.server }));
case 'LIKE_ROLLBACK': return updatePost(state, id, (p) => ({ ...p, liked: p.server.liked }));

export const likeCount = (post) =>
  post.server.likes + (post.liked === post.server.liked ? 0 : post.liked ? 1 : -1);
```

Challenge 2: Rapid clicks and out-of-order responses

Problem: User clicks like, unlike, like in 300 ms. Three requests fly; they can return in any order. If "unlike" returns last, the UI (and maybe the server) ends unliked while the user's last click was like. If the second request fails while the third succeeds, which one rolls back?

Solution: Single-flight + coalescing per post. Keep the user's latest intent in a ref. Only one request per post is on the wire. When it returns, if the intent changed meanwhile, send one more request with the *current* intent. Responses can't arrive out of order because there is never more than one. Five fast clicks cost at most two requests; an even number of clicks often costs one.

```
async function syncLike(id) {
  inFlight.current.add(id);
  try {
    let server;
    do {
      server = await api.setLike(id, intent.current[id]);
      dispatch({ type: 'LIKE_SYNC', id, server });
    } while (server.liked !== intent.current[id]); // user clicked again meanwhile
  } catch {
    delete intent.current[id];
    dispatch({ type: 'LIKE_ROLLBACK', id });
    notify("Couldn't update like. Please try again.");
  } finally {
    inFlight.current.delete(id);
  }
}

function toggleLike(post) {
  const liked = !(intent.current[post.id] ?? post.liked);
  intent.current[post.id] = liked;
  dispatch({ type: 'LIKE_SET', id: post.id, liked });
  if (!inFlight.current.has(post.id)) syncLike(post.id);
}
```

Alternative to mention: a request sequence number per post, ignoring responses older than the latest. It works, but rollback gets tricky (which confirmed value is the latest?), so single-flight is simpler to reason about.

Challenge 3: Optimistic items that don't have an id yet

Problem: A new comment needs a React key and must be matched to the server response, but the server creates the id. Using the array index breaks when a comment above is discarded. Removing a failed comment silently loses what the user typed. A retry after a timeout could create a duplicate on the server.

Solution: Generate a `clientId` on the client, use it as the key, the match key and the idempotency key. Track `status` per comment. On success replace the temp item with the server item; on failure keep it with Retry / Discard.

```
function addComment(postId, text) {
  const clientId = crypto.randomUUID();
  dispatch({ type: 'COMMENT_ADD', id: postId, clientId, text }); // status: 'pending'
  sendComment(postId, clientId, text);
```

```
}

async function sendComment(postId, clientId, text) {
  try {
    const comment = await api.addComment(postId, clientId, text);
    dispatch({ type: 'COMMENT_OK', id: postId, clientId, comment });
  } catch {
    dispatch({ type: 'COMMENT_FAIL', id: postId, clientId });
  }
}
```

Challenge 4: Telling the user without blocking them

Problem: A rollback that happens silently looks like a bug ("I liked it, why did it unlike?"). A modal is too heavy. A toast whose timer depends on an inline `onClose` gets reset on every re-render and never hides.

Solution: A toast in an always-mounted `aria-live="polite"` region, `notice` stored as `{ id, text }` so the same text twice restarts the timer, and a stable `clearNotice` from `useCallback`.

```
const notify = (text) => setNotice({ id: Date.now(), text });
const clearNotice = useCallback(() => setNotice(null), []);

useEffect(() => {
  if (!notice) return;
  const t = setTimeout(onClose, 3000);
  return () => clearTimeout(t);
}, [notice, onClose]);
```

---

8. Implement in the correct order

#	Step	Working result
1	Fake API with latency, `config.failRate`	Promises that resolve/reject like a server
2	Reducer with `LOAD_*`, `useFeed` loads on mount	Loading → feed rendered
3	`PostCard` with static like count and comments	Full read-only feed
4	Pessimistic like (await, then update)	Works, but slow: the baseline
5	Make it optimistic: `LIKE_SET` + `server` field + derived count	Instant like
6	`LIKE_ROLLBACK` on failure + toast	Rollback works
7	Single-flight + coalescing with `intent` / `inFlight` refs	Rapid clicks stay correct
8	Optimistic comment with `clientId`, `pending` → `sent`	Instant comment
9	`failed` status, Retry, Discard	No lost comments

#	Step	Working result
10	Load error + Retry, empty state	All page states
11	"Simulate failures" toggle, a11y attributes	Demo-ready

9. Cover all states and edge cases

State / case	Handling
Initial	`status: 'loading'`, "Loading posts…"
Load error	`role="alert"` message + Retry (`LOAD_START` then fetch)
Empty feed	"No posts yet."
Post with no comments	"No comments yet."
Like pending	Heart and count already updated, "saving…"
Like failed	Rollback to `server.liked`, toast
Like failed after a coalesced second request	Rollback to the value the first request confirmed (`server` was updated by `LIKE_SYNC`)
Like → unlike before response	Loop sees `server.liked !== intent`, sends `false`; final state correct
Like → unlike → like before response	Loop sees `server.liked === intent`, stops: 1 request total
Comment pending	Faded, "Sending…"
Comment failed	Kept, "Failed", Retry / Discard
Retry after timeout that actually succeeded	Server de-duplicates by `clientId`
Empty / whitespace comment	Trimmed; submit disabled
Very long comment	`maxLength={500}`
Same error twice in a row	`notice.id` changes, toast restarts its timer
Server mutated but response lost	UI rolls back; idempotent retry or next refetch fixes it

10. Final structure

```
src/
├── api/
│   └── fakeApi.js          # in-memory server, latency, failRate, idempotent endpoints
├── state/
│   └── feedReducer.js      # pure transitions + likeCount (derived)
├── hooks/
│   └── useFeed.js          # requests, coalescing, rollback, toast notice
```

```
├── components/
│   ├── PostCard.jsx
│   ├── LikeButton.jsx
│   ├── CommentList.jsx
│   ├── CommentForm.jsx
│   └── Toast.jsx
├── App.jsx
├── main.jsx                # unchanged Vite entry
└── index.css               # @import "tailwindcss";
```

---

11. Full working code

`src/api/fakeApi.js`

```
// Fake backend: in-memory DB, random latency, failure rate the UI can toggle.
const db = {
  posts: [
    { id: 'p1', author: 'Asha', text: 'Shipped optimistic likes today 🚀', liked: false, likes: 12,
      comments: [{ id: 'c1', clientId: 'seed-1', text: 'Nice work!' }] },
    { id: 'p2', author: 'Ravi', text: 'Rollback is the hard part of optimistic UI.', liked: true,
likes: 3, comments: [] },
  ],
};

export const config = { failRate: 0.3 };

const wait = () => new Promise((resolve) => setTimeout(resolve, 400 + Math.random() * 800));

async function request(handler) {
  await wait();
  if (Math.random() < config.failRate) throw new Error('Network error');
  return structuredClone(handler()); // never hand out references to the "server" data
}

const findPost = (postId) => {
  const post = db.posts.find((p) => p.id === postId);
  if (!post) throw new Error('Post not found');
  return post;
};

export const api = {
  getPosts: () => request(() => db.posts),

  // Idempotent: sends the desired state, not "toggle", so retries and repeats are safe.
```

```
  setLike: (postId, liked) =>
    request(() => {
      const post = findPost(postId);
      if (post.liked !== liked) {
        post.liked = liked;
        post.likes += liked ? 1 : -1;
      }
      return { liked: post.liked, likes: post.likes };
    }),

  // clientId makes a retried comment a no-op instead of a duplicate.
  addComment: (postId, clientId, text) =>
    request(() => {
      const post = findPost(postId);
      let comment = post.comments.find((c) => c.clientId === clientId);
      if (!comment) {
        comment = { id: crypto.randomUUID(), clientId, text };
        post.comments.push(comment);
      }
      return comment;
    }),
};
```

src/state/feedReducer.js

```
export const initialState = { status: 'loading', posts: [], error: null };

// post.server = last state the server confirmed; post.liked = what the user sees right now.
export const toPost = (p) => ({
  id: p.id,
  author: p.author,
  text: p.text,
  liked: p.liked,
  server: { liked: p.liked, likes: p.likes },
  comments: p.comments.map((c) => ({ ...c, status: 'sent' })),
});

// Count is derived: confirmed count adjusted by the not-yet-confirmed like.
export const likeCount = (post) =>
  post.server.likes + (post.liked === post.server.liked ? 0 : post.liked ? 1 : -1);

const updatePost = (state, id, fn) => ({
  ...state,
  posts: state.posts.map((p) => (p.id === id ? fn(p) : p)),
});
```

```
const updateComment = (post, clientId, fn) => ({
  ...post,
  comments: post.comments.map((c) => (c.clientId === clientId ? fn(c) : c)),
});

export function feedReducer(state, action) {
  switch (action.type) {
    case 'LOAD_START':
      return { ...state, status: 'loading', error: null };
    case 'LOAD_OK':
      return { status: 'ready', posts: action.posts.map(toPost), error: null };
    case 'LOAD_FAIL':
      return { ...state, status: 'error', error: action.error };

    case 'LIKE_SET': // optimistic
      return updatePost(state, action.id, (p) => ({ ...p, liked: action.liked }));
    case 'LIKE_SYNC': // server confirmed; keep the optimistic liked, it may already be newer
      return updatePost(state, action.id, (p) => ({ ...p, server: action.server }));
    case 'LIKE_ROLLBACK':
      return updatePost(state, action.id, (p) => ({ ...p, liked: p.server.liked }));

    case 'COMMENT_ADD': // optimistic
      return updatePost(state, action.id, (p) => ({
        ...p,
        comments: [...p.comments, { id: null, clientId: action.clientId, text: action.text, status:
'pending' }],
      }));
    case 'COMMENT_OK':
      return updatePost(state, action.id, (p) =>
        updateComment(p, action.clientId, () => ({ ...action.comment, status: 'sent' })));
    case 'COMMENT_FAIL':
      return updatePost(state, action.id, (p) =>
        updateComment(p, action.clientId, (c) => ({ ...c, status: 'failed' })));
    case 'COMMENT_RETRY':
      return updatePost(state, action.id, (p) =>
        updateComment(p, action.clientId, (c) => ({ ...c, status: 'pending' })));
    case 'COMMENT_DISCARD':
      return updatePost(state, action.id, (p) => ({
        ...p,
        comments: p.comments.filter((c) => c.clientId !== action.clientId),
      }));

    default:
      return state;
  }
}
```

`src/hooks/useFeed.js`

```
import { useCallback, useEffect, useReducer, useRef, useState } from 'react';
import { api } from '../api/fakeApi';
import { feedReducer, initialState } from '../state/feedReducer';

export default function useFeed() {
  const [state, dispatch] = useReducer(feedReducer, initialState);
  const [notice, setNotice] = useState(null);
  const intent = useRef({}); // postId -> liked value the user wants
  const inFlight = useRef(new Set()); // postIds with a like request on the wire

  const notify = (text) => setNotice({ id: Date.now(), text });
  const clearNotice = useCallback(() => setNotice(null), []);

  const load = useCallback(async () => {
    try {
      dispatch({ type: 'LOAD_OK', posts: await api.getPosts() });
    } catch (e) {
      dispatch({ type: 'LOAD_FAIL', error: e.message });
    }
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  const reload = () => {
    dispatch({ type: 'LOAD_START' });
    load();
  };

  // One request per post at a time; clicks made meanwhile are coalesced into the next request.
  async function syncLike(id) {
    inFlight.current.add(id);
    try {
      let server;
      do {
        server = await api.setLike(id, intent.current[id]);
        dispatch({ type: 'LIKE_SYNC', id, server });
      } while (server.liked !== intent.current[id]);
    } catch {
      delete intent.current[id];
      dispatch({ type: 'LIKE_ROLLBACK', id });
      notify("Couldn't update like. Please try again.");
    } finally {
      inFlight.current.delete(id);
```

```
    }
  }

  function toggleLike(post) {
    const liked = !(intent.current[post.id] ?? post.liked);
    intent.current[post.id] = liked;
    dispatch({ type: 'LIKE_SET', id: post.id, liked });
    if (!inFlight.current.has(post.id)) syncLike(post.id);
  }

  async function sendComment(postId, clientId, text) {
    try {
      const comment = await api.addComment(postId, clientId, text);
      dispatch({ type: 'COMMENT_OK', id: postId, clientId, comment });
    } catch {
      dispatch({ type: 'COMMENT_FAIL', id: postId, clientId });
      notify("Comment wasn't posted. You can retry it.");
    }
  }

  function addComment(postId, text) {
    const clientId = crypto.randomUUID();
    dispatch({ type: 'COMMENT_ADD', id: postId, clientId, text });
    sendComment(postId, clientId, text);
  }

  function retryComment(postId, comment) {
    dispatch({ type: 'COMMENT_RETRY', id: postId, clientId: comment.clientId });
    sendComment(postId, comment.clientId, comment.text);
  }

  const discardComment = (postId, clientId) =>
    dispatch({ type: 'COMMENT_DISCARD', id: postId, clientId });

  return { ...state, notice, clearNotice, reload, toggleLike, addComment, retryComment, discardComment
};
}
```

src/components/LikeButton.jsx

```
import { likeCount } from '../state/feedReducer';

export default function LikeButton({ post, onToggle }) {
  const count = likeCount(post);
  const saving = post.liked !== post.server.liked;
```

```
  // Never disabled while saving: instant feedback is the point; extra clicks are coalesced.
  return (
    <button
      type="button"
      aria-pressed={post.liked}
      aria-label={`${post.liked ? 'Unlike' : 'Like'} post, ${count} likes`}
      onClick={() => onToggle(post)}
      className={`rounded-full border px-3 py-1 text-sm ${
        post.liked ? 'border-rose-300 bg-rose-50 text-rose-600' : 'border-gray-300 text-gray-700'
      }`}
    >
      <span aria-hidden="true">{post.liked ? '♥' : '♡'}</span> {count}
      {saving && <span className="ml-1 text-xs text-gray-400">saving…</span>}
    </button>
  );
}
```

src/components/CommentList.jsx

```
export default function CommentList({ comments, onRetry, onDiscard }) {
  if (comments.length === 0) return <p className="text-sm text-gray-400">No comments yet.</p>;

  return (
    <ul className="space-y-1">
      {comments.map((c) => (
        <li
          key={c.clientId ?? c.id}
          className={`flex items-center gap-2 text-sm ${c.status === 'pending' ? 'opacity-60' : ''}`}
        >
          <span className="flex-1">{c.text}</span>
          {c.status === 'pending' && <span className="text-xs text-gray-400">Sending…</span>}
          {c.status === 'failed' && (
            <>
              <span className="text-xs text-red-600">Failed</span>
              <button type="button" className="text-xs text-blue-600 underline" onClick={() =>
onRetry(c)}>
                Retry
              </button>
              <button type="button" className="text-xs text-gray-500 underline" onClick={() =>
onDiscard(c.clientId)}>
                Discard
              </button>
            </>
          )}
        </li>
      ))}
```

```
    </ul>
  );
}
```

src/components/CommentForm.jsx

```
import { useState } from 'react';

export default function CommentForm({ onSubmit }) {
  const [text, setText] = useState('');
  const trimmed = text.trim();

  function handleSubmit(e) {
    e.preventDefault();
    if (!trimmed) return;
    onSubmit(trimmed);
    setText(''); // optimistic: clear at once, the comment is already in the list
  }

  return (
    <form onSubmit={handleSubmit} className="flex gap-2">
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        maxLength={500}
        placeholder="Add a comment…"
        aria-label="Add a comment"
        className="flex-1 rounded border border-gray-300 px-2 py-1 text-sm"
      />
      <button type="submit" disabled={!trimmed} className="rounded bg-blue-600 px-3 py-1 text-sm text-
white disabled:opacity-40">
        Post
      </button>
    </form>
  );
}
```

src/components/PostCard.jsx

```
import LikeButton from './LikeButton';
import CommentList from './CommentList';
import CommentForm from './CommentForm';

export default function PostCard({ post, onToggleLike, onAddComment, onRetryComment, onDiscardComment
}) {
```

```
  return (
    <article className="space-y-3 rounded-lg border border-gray-200 bg-white p-4 shadow-sm">
      <header className="font-semibold">{post.author}</header>
      <p>{post.text}</p>
      <LikeButton post={post} onToggle={onToggleLike} />
      <CommentList
        comments={post.comments}
        onRetry={(c) => onRetryComment(post.id, c)}
        onDiscard={(clientId) => onDiscardComment(post.id, clientId)}
      />
      <CommentForm onSubmit={(text) => onAddComment(post.id, text)} />
    </article>
  );
}
```

src/components/Toast.jsx

```
import { useEffect } from 'react';

export default function Toast({ notice, onClose }) {
  useEffect(() => {
    if (!notice) return;
    const t = setTimeout(onClose, 3000);
    return () => clearTimeout(t);
  }, [notice, onClose]); // notice is a new object per message, so a repeat message restarts the timer

  // The live region stays mounted so screen readers announce text added to it.
  return (
    <div role="status" aria-live="polite" className="fixed bottom-4 left-1/2 -translate-x-1/2">
      {notice && <p className="rounded bg-gray-900 px-4 py-2 text-sm text-white">{notice.text}</p>}
    </div>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import { config } from './api/fakeApi';
import useFeed from './hooks/useFeed';
import PostCard from './components/PostCard';
import Toast from './components/Toast';

export default function App() {
  const feed = useFeed();
  const [failures, setFailures] = useState(config.failRate > 0);
```

```
  function toggleFailures(e) {
    config.failRate = e.target.checked ? 0.3 : 0;
    setFailures(e.target.checked);
  }

  return (
    <main className="mx-auto max-w-xl space-y-4 p-4">
      <header className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">Feed</h1>
        <label className="flex items-center gap-2 text-sm">
          <input type="checkbox" checked={failures} onChange={toggleFailures} />
          Simulate failures (30%)
        </label>
      </header>

      {feed.status === 'loading' && <p aria-busy="true">Loading posts…</p>}

      {feed.status === 'error' && (
        <div role="alert" className="space-y-2">
          <p className="text-red-600">Couldn't load posts: {feed.error}</p>
          <button type="button" onClick={feed.reload} className="rounded bg-gray-800 px-3 py-1 text-
white">
            Retry
          </button>
        </div>
      )}

      {feed.status === 'ready' && feed.posts.length === 0 && <p>No posts yet.</p>}

      {feed.status === 'ready' &&
        feed.posts.map((post) => (
          <PostCard
            key={post.id}
            post={post}
            onToggleLike={feed.toggleLike}
            onAddComment={feed.addComment}
            onRetryComment={feed.retryComment}
            onDiscardComment={feed.discardComment}
          />
        ))}

      <Toast notice={feed.notice} onClose={feed.clearNotice} />
    </main>
  );
}
```

Self-check of the pure logic (`node check.mjs`)

```
import assert from 'node:assert/strict';
import { feedReducer, initialState, likeCount } from './src/state/feedReducer.js';
import { api, config } from './src/api/fakeApi.js';

config.failRate = 0;
let s = feedReducer(initialState, { type: 'LOAD_OK', posts: await api.getPosts() });
const p = () => s.posts[0];
assert.equal(likeCount(p()), 12);

s = feedReducer(s, { type: 'LIKE_SET', id: 'p1', liked: true });
assert.equal(likeCount(p()), 13);
s = feedReducer(s, { type: 'LIKE_ROLLBACK', id: 'p1' });
assert.equal(p().liked, false);
assert.equal(likeCount(p()), 12);

// confirmed while user already flipped again: count stays consistent
s = feedReducer(s, { type: 'LIKE_SET', id: 'p1', liked: false });
s = feedReducer(s, { type: 'LIKE_SYNC', id: 'p1', server: { liked: true, likes: 13 } });
assert.equal(likeCount(p()), 12);

// idempotent setLike
assert.deepEqual(await api.setLike('p2', true), { liked: true, likes: 3 });
assert.deepEqual(await api.setLike('p2', false), { liked: false, likes: 2 });

// comments: add -> fail -> retry -> ok; retry doesn't duplicate on the server
s = feedReducer(s, { type: 'COMMENT_ADD', id: 'p1', clientId: 'x', text: 'hi' });
assert.equal(p().comments.at(-1).status, 'pending');
s = feedReducer(s, { type: 'COMMENT_FAIL', id: 'p1', clientId: 'x' });
assert.equal(p().comments.at(-1).status, 'failed');
s = feedReducer(s, { type: 'COMMENT_RETRY', id: 'p1', clientId: 'x' });
const c = await api.addComment('p1', 'x', 'hi');
await api.addComment('p1', 'x', 'hi');
s = feedReducer(s, { type: 'COMMENT_OK', id: 'p1', clientId: 'x', comment: c });
assert.equal(p().comments.at(-1).status, 'sent');
assert.ok(p().comments.at(-1).id);
assert.equal((await api.getPosts())[0].comments.length, 2);
s = feedReducer(s, { type: 'COMMENT_DISCARD', id: 'p1', clientId: 'x' });
assert.equal(p().comments.length, 1);
console.log('all checks passed');
```

12. Wrap-up

Edge cases handled

- Rollback to the server-confirmed value, never a stale snapshot.
- Rapid clicks coalesced; no out-of-order responses; final state = final click.
- Optimistic comments with client ids: no index keys, no duplicates on retry, no lost text.
- Loading, error with retry, empty feed, empty comments, whitespace input, repeated errors.
- Accessible: `aria-pressed` toggle, labelled input, `aria-live` toast, `role="alert"` on load failure.

Trade-offs to mention

- **Optimistic vs pessimistic:** optimistic is right for cheap, likely-to-succeed, reversible actions (like, comment). For payments, deletes of important data or anything with side effects the user can't undo, wait for the server.
- **Single-flight vs sequence numbers vs `AbortController`:** single-flight is simplest and saves requests; abort doesn't stop a request that already reached the server.
- **Keep failed comments vs remove them:** keeping them is friendlier but adds UI states.
- **`useReducer` + custom hook vs React 19 `useOptimistic`:** `useOptimistic` reverts automatically when the transition ends, which is great for simple cases, but per-item statuses, retry and coalescing are easier to control with an explicit reducer.
- **Fake API with module-level config:** fine for a demo; a real app passes it in or uses a mock server (MSW).

With more time

- Tests: React Testing Library with fake timers and a controllable API mock (resolve/reject on demand).
- Use TanStack Query `onMutate` / `onError` / `onSettled` for cache-level optimistic updates and refetch after settle.
- Retry with exponential backoff for likes; offline queue (persist pending mutations, replay on `online`).
- Real-time updates (WebSocket) merged into `server` without clobbering pending optimistic values.
- Undo toast ("Comment posted · Undo"), pagination / virtualization for long feeds.

---

13. Counter questions the interviewer may ask

Requirements

Q1. When should you NOT use optimistic UI?

When failure is likely, expensive, or irreversible: payments, sending money, deleting accounts, actions with external side effects (emails), or when the server computes the result (price, rank). Use it for cheap, frequent, reversible actions that almost always succeed.

Q2. Why not disable the like button while the request is in flight?

It defeats the purpose: the user feels the latency again, and quick undo ("oops, didn't mean to like") becomes impossible. Coalescing makes extra clicks cheap and safe instead.

Q3. Why does a failed comment stay instead of disappearing like a failed like?

A like is one click to redo; a comment is user-written content. Removing it silently loses work. Keeping it with Retry / Discard respects the user's effort.

State design

Q4. Why store `server` next to `liked` instead of a snapshot taken before the request?

A snapshot can be stale by the time the request fails (other updates arrived meanwhile) and gets confusing with overlapping requests. The confirmed value is always the latest truth, so rollback is one assignment.

Q5. Why derive the like count instead of storing an optimistic count?

Two pieces of state that must agree will eventually disagree. Derived from `server.likes` and `liked !== server.liked`, the count can only ever be off by exactly the pending change.

Q6. Why are `intent` and `inFlight` refs and not state?

They coordinate async requests, not rendering. They must be read with their latest value inside an `await` loop (state would be a stale closure) and changing them shouldn't cause re-renders.

Q7. Why `useReducer` and not several `useState`s?

Many transitions touch nested items (post → comment). A pure reducer keeps every transition in one place, is easy to unit test without React, and lets async code dispatch intent without knowing the state shape.

Challenge: rollback and races

Q8. User clicks like, unlike, like very fast. Walk me through it.

Click 1 sets intent `true`, sends request A. Clicks 2 and 3 only flip the UI and intent (`false`, then `true`); no new requests because A is in flight. A returns `liked: true` = intent, the loop stops. One request, correct result.

Q9. What if the first request succeeds but the coalesced second one fails?

`LIKE_SYNC` already stored the first response in `server`, so rollback goes to that confirmed value, which matches the server. The user sees a toast and can click again.

Q10. What if the request times out on the client but the server actually applied it?

The UI rolls back and is now wrong. Idempotent endpoints make it safe: the user clicks again and `setLike(true)` is a no-op on the server, or the next refetch corrects it. For comments, `clientId` de-duplicates a retry.

Q11. Why not use `AbortController` to cancel the older request?

Aborting stops the client from reading the response; it does not stop the server from applying it. You still need idempotency and a rule for which response wins. It's useful for GETs, less for mutations.

Challenge: comments

Q12. Why not use the array index as the key for pending comments?

If a comment above is discarded, indexes shift and React reuses the wrong DOM node and state. `clientId` is stable from the moment the comment is created.

Q13. How does Retry avoid creating two comments on the server?

It resends the same `clientId`. The server returns the existing comment if it already has that `clientId`. In real APIs this is the `Idempotency-Key` header.

Implementation steps

Q14. How would you do this with React 19 `useOptimistic`?

Wrap the mutation in a transition (form action or `startTransition`), call `addOptimistic(newComment)` and await the API. React shows the optimistic value during the transition and drops it when the transition ends; you then set the real state on success. Failed-but-kept comments and coalescing still need your own state.

```
const [optimistic, addOptimistic] = useOptimistic(comments, (list, c) => [...list, { ...c, status:
'pending' }]);
```

Q15. How would you do it with TanStack Query?

`useMutation` with `onMutate` (cancel queries, snapshot cache, `setQueryData` optimistic), `onError` (restore snapshot), `onSettled` (`invalidateQueries` to refetch server truth). The refetch on settle fixes any drift.

Q16. What if the component unmounts while a request is in flight?

Here the state lives in `useFeed` at the app root, so it never unmounts. If it lived in a page, keep the mutation in a store/query cache outside the component, or ignore late dispatches; React 18+ no longer warns about setState after unmount.

Performance

Q17. What if the feed has 10,000 posts?

`posts.map` on every like is O(n) and re-renders every `PostCard`. Normalize to `byId` + `ids`, wrap `PostCard` in `React.memo` with stable callbacks, and virtualize the list (react-window). Paginate with infinite scroll instead of loading everything.

Q18. What if the API is very slow (5 s)?

Optimistic UI hides it for likes. Show a subtle pending indicator ("saving…", faded comment) so users don't leave thinking it's saved; warn on `beforeunload` if mutations are pending; add a timeout so a hung request eventually fails and rolls back.

Trade-offs and "what if"

Q19. What if another user's like arrives over a WebSocket while my like is pending?

Update only `server.likes`, keep my optimistic `liked`. Because the count is derived, the UI shows the new total plus my pending change, and rollback still lands on correct data.

Q20. What if the user goes offline?

Queue mutations (in memory or IndexedDB) with their idempotency keys, keep the optimistic UI with a "will sync" hint, replay them on the `online` event, and roll back the ones the server rejects.

Q21. How would you test the rollback?

Mock the API with promises you resolve or reject by hand. Assert: after click, ♥ and count+1 are visible before resolving; after reject, ♡ and original count are back and the toast is shown. For coalescing, click three times, resolve once, and assert the mock was called once.